AyuraBlend - Complete E-Commerce Platform Architecture Documentation

Document Version: 1.0
Date: April 2026
Project Type: Full-Stack E-Commerce Platform
Tech Stack: React + Node.js + MongoDB + Razorpay

Table of Contents

1. [System Architecture](#1-system-architecture)
2. [Folder Structure](#2-folder-structure)
3. [API Endpoints](#3-api-endpoints)
4. [Database Schemas](#4-database-schemas)
5. [Data Flow](#5-data-flow)
6. [React Component Structure](#6-react-component-structure)
7. [Step-by-Step Build Plan](#7-step-by-step-build-plan)
8. [Key Implementation Examples](#8-key-implementation-examples)
9. [Environment Variables](#9-environment-variables)
10. [Deployment Checklist](#10-deployment-checklist)

---

1. System Architecture

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│                              CLIENT (Browser)                               │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    │ HTTPS
                                    ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│                              Frontend (React + Vite)                        │
│                                                                             │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐       │
│   │                  │   │                  │   │                  │       │
│   │   Product        │   │   Cart           │   │   Checkout       │   │   Orders        │   │
│   │   Listing        │   │                  │   │                  │       │
│   │                  │   │                  │   │                  │       │
│   └──────────────────┘   └──────────────────┘   └──────────────────┘       │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

```
│  REST API
┌──────────────────────────────────────────────▼──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  Backend (Node.js + Express)  │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐   │
│   │  Auth  │   │  Products  │   │  Orders  │   │  Payment  │   │
│   │  Middle-  │   │  Routes  │   │  Routes  │   │  Webhooks  │   │
│   │  ware  │   │   │   │   │   │   │   │   │   │
│   └──────────────────┘   └──────────────────┘   └──────────────────┘   │
└──────────────────────────────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
┌──────────────────────────────────────┼──────────────────────────────────────┬──────────────────────────────────────┐
│   │   │   │
┌──────▼──────┐   ┌──────▼──────┐   ┌──────▼──────┐   ┌──────▼──────┐
│  MongoDB  │   │ Cloudinary │   │ Razorpay │   │ WhatsApp Cloud API │
│  (Atlas)  │   │  (Images)  │   │ Payment  │   │  (Fallback)  │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────────────────┘
```

Architecture Overview

Frontend (React + Vite + Tailwind CSS)
- Single Page Application with client-side routing
- State management using React Context API
- Persistent cart storage using localStorage
- Responsive design with Tailwind CSS

Backend (Node.js + Express)
- RESTful API architecture
- MVC pattern implementation
- JWT-based authentication (optional)
- Rate limiting and security middleware

Database (MongoDB Atlas)
- Cloud-hosted NoSQL database
- Indexed collections for performance
- Automatic backups and scaling

Third-Party Services
- **Cloudinary**: Image optimization and CDN delivery
- **Razorpay**: Primary payment gateway with UPI/Cards/NetBanking
- **PayPal**: Optional secondary payment method
- **WhatsApp Cloud API**: Order fallback and customer communication

---

2. Folder Structure

Frontend Structure (`frontend/`)

```
frontend/
 ├── public/
 |   ├── favicon.ico
 |   └── logo.png
 ├── src/
 |   ├── components/
 |   |   ├── common/
 |   |   |   ├── Navbar.jsx
 |   |   |   ├── Footer.jsx
 |   |   |   ├── Loader.jsx
```

```
│   │   │   └── ErrorMessage.jsx
│   │   ├── products/
│   │   │   ├── ProductCard.jsx
│   │   │   ├── ProductList.jsx
│   │   │   ├── ProductDetail.jsx
│   │   │   └── ProductFilter.jsx
│   │   ├── cart/
│   │   │   ├── CartIcon.jsx
│   │   │   ├── CartDrawer.jsx
│   │   │   └── CartItem.jsx
│   │   └── checkout/
│   │   ├── CheckoutForm.jsx
│   │   ├── OrderSummary.jsx
│   │   ├── PaymentButton.jsx
│   │   └── WhatsAppOrder.jsx
│   ├── pages/
│   │   ├── HomePage.jsx
│   │   ├── ProductsPage.jsx
│   │   ├── ProductDetailPage.jsx
│   │   ├── CartPage.jsx
│   │   ├── CheckoutPage.jsx
│   │   ├── OrderSuccessPage.jsx
│   │   └── OrderFailurePage.jsx
│   ├── context/
│   │   ├── CartContext.jsx
│   │   └── OrderContext.jsx
│   ├── services/
│   │   ├── api.js
│   │   ├── productService.js
```

```
│   │   ├── orderService.js
│   │   └── paymentService.js
│   ├── hooks/
│   │   ├── useCart.js
│   │   └── useLocalStorage.js
│   ├── utils/
│   │   ├── constants.js
│   │   ├── formatters.js
│   │   └── validators.js
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── .env
├── .gitignore
├── package.json
├── vite.config.js
└── tailwind.config.js
```

Backend Structure (`backend/`)

```
backend/
├── src/
│   ├── models/
│   │   ├── Product.js
│   │   ├── Order.js
│   │   └── User.js
│   ├── controllers/
│   │   ├── productController.js
│   │   ├── orderController.js
```

```
│   │   └── paymentController.js
│   ├── routes/
│   │   ├── productRoutes.js
│   │   ├── orderRoutes.js
│   │   └── paymentRoutes.js
│   ├── middleware/
│   │   ├── errorHandler.js
│   │   ├── validation.js
│   │   ├── auth.js
│   │   └── rateLimiter.js
│   ├── services/
│   │   ├── cloudinaryService.js
│   │   ├── razorpayService.js
│   │   ├── whatsappService.js
│   │   └── emailService.js
│   ├── utils/
│   │   ├── db.js
│   │   ├── logger.js
│   │   └── helpers.js
│   └── app.js
├── .env
├── .gitignore
├── package.json
└── server.js
```

---

3. API Endpoints

Products API

Method	Endpoint	Description	Auth Required
GET	`/api/products`	Get all products with pagination & filters	No
GET	`/api/products/:id`	Get single product by ID	No
GET	`/api/products/slug/:slug`	Get product by slug	No
GET	`/api/products/category/:category`	Get products by category	No
POST	`/api/products`	Create new product	Yes (Admin)
PUT	`/api/products/:id`	Update product	Yes (Admin)
DELETE	`/api/products/:id`	Delete product	Yes (Admin)

Query Parameters for GET /api/products:
- `page` (number, default: 1) - Pagination page
- `limit` (number, default: 12) - Items per page
- `category` (string) - Filter by category
- `minPrice` (number) - Minimum price filter
- `maxPrice` (number) - Maximum price filter
- `sort` (string) - Sort by: 'price_asc', 'price_desc', 'newest', 'popular'
- `search` (string) - Search in product name/description

Orders API

Method	Endpoint	Description	Auth Required
POST	`/api/orders`	Create new order	No
GET	`/api/orders/:id`	Get order by ID	Yes (Owner/Admin)
GET	`/api/orders/phone/:phone`	Get orders by phone number	Yes (Owner)
PUT	`/api/orders/:id/status`	Update order status	Yes (Admin)
POST	`/api/orders/:id/whatsapp`	Send WhatsApp fallback	No
GET	`/api/orders/track/:orderId`	Track order status	No

Payment API

Method	Endpoint	Description

POST	`/api/payment/create-order`	Create Razorpay order
POST	`/api/payment/verify`	Verify payment signature
POST	`/api/payment/webhook`	Razorpay webhook endpoint
POST	`/api/payment/paypal/create`	Create PayPal order
POST	`/api/payment/paypal/capture`	Capture PayPal payment

---

4. Database Schemas

Product Schema

```javascript
// models/Product.js
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product name is required'],
    trim: true,
    maxlength: [100, 'Product name cannot exceed 100 characters']
  },
  slug: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },
  description: {
    type: String,
    required: [true, 'Product description is required'],
    minlength: [20, 'Description must be at least 20 characters']
  },
  shortDescription: {
    type: String,
    maxlength: [200, 'Short description cannot exceed 200 characters']
```

```
  },
  price: {
    type: Number,
    required: [true, 'Product price is required'],
    min: [0, 'Price cannot be negative']
  },
  compareAtPrice: {
    type: Number,
    min: [0, 'Compare at price cannot be negative']
  },
  category: {
    type: String,
    enum: ['essential-oils', 'herbal-teas', 'wellness-kits', 'diffusers',
'accessories'],
    required: [true, 'Product category is required']
  },
  images: [{
    url: {
      type: String,
      required: true
    },
    publicId: {
      type: String,
      required: true
    },
    alt: {
      type: String,
      default: ''
    }
  }],
  stock: {
    type: Number,
    required: true,
    min: [0, 'Stock cannot be negative'],
    default: 0
  },
  isAvailable: {
    type: Boolean,
```

```
    default: true
  },
  attributes: {
    weight: String,
    dimensions: String,
    ingredients: [String],
    benefits: [String],
    howToUse: String,
    brand: String,
    origin: String
  },
  rating: {
    type: Number,
    min: 0,
    max: 5,
    default: 0
  },
  reviewCount: {
    type: Number,
    default: 0
  },
  metaTitle: String,
  metaDescription: String,
  createdAt: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true,
  toJSON: { virtuals: true },
  toObject: { virtuals: true }
});

// Virtual for discount percentage
```

```
productSchema.virtual('discountPercentage').get(function() {
  if (this.compareAtPrice && this.compareAtPrice > this.price) {
    return Math.round(((this.compareAtPrice - this.price) /
this.compareAtPrice) * 100);
  }
  return 0;
});

// Indexes for better query performance
productSchema.index({ name: 'text', description: 'text' });
productSchema.index({ category: 1, price: 1 });
productSchema.index({ slug: 1 });

module.exports = mongoose.model('Product', productSchema);


####order schema
// models/Order.js
const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({
  orderId: {
    type: String,
    required: true,
    unique: true,
    default: function() {
      return 'AYURA-' + Date.now().toString(36).toUpperCase() +
             Math.random().toString(36).substr(2, 4).toUpperCase();
    }
  },
  customer: {
    name: {
      type: String,
      required: [true, 'Customer name is required'],
      trim: true
    },
    email: {
      type: String,
```

```
      required: [true, 'Email is required'],
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email']
    },
    phone: {
      type: String,
      required: [true, 'Phone number is required'],
      match: [/^[0-9]{10}$/, 'Please enter a valid 10-digit phone
number']
    },
    address: {
      line1: {
        type: String,
        required: [true, 'Address line 1 is required']
      },
      line2: String,
      city: {
        type: String,
        required: [true, 'City is required']
      },
      state: {
        type: String,
        required: [true, 'State is required']
      },
      pincode: {
        type: String,
        required: [true, 'Pincode is required'],
        match: [/^[0-9]{6}$/, 'Please enter a valid 6-digit pincode']
      },
      country: {
        type: String,
        default: 'India'
      }
    }
  },
  items: [{
    productId: {
      type: mongoose.Schema.Types.ObjectId,
```

```
      ref: 'Product',
      required: true
    },
    name: {
      type: String,
      required: true
    },
    slug: String,
    price: {
      type: Number,
      required: true,
      min: 0
    },
    quantity: {
      type: Number,
      required: true,
      min: 1
    },
    image: String
  }],
  summary: {
    subtotal: {
      type: Number,
      required: true,
      min: 0
    },
    shipping: {
      type: Number,
      default: 0,
      min: 0
    },
    tax: {
      type: Number,
      default: 0,
      min: 0
    },
    discount: {
      type: Number,
```

```
      default: 0,
      min: 0
    },
    total: {
      type: Number,
      required: true,
      min: 0
    }
  },
  payment: {
    method: {
      type: String,
      enum: ['razorpay', 'paypal', 'whatsapp', 'cod'],
      required: true
    },
    status: {
      type: String,
      enum: ['pending', 'success', 'failed', 'refunded'],
      default: 'pending'
    },
    razorpayOrderId: String,
    razorpayPaymentId: String,
    razorpaySignature: String,
    paypalOrderId: String,
    paypalCaptureId: String,
    paidAt: Date
  },
  status: {
    type: String,
    enum: ['pending', 'confirmed', 'processing', 'shipped', 'delivered',
'cancelled'],
    default: 'pending'
  },
  whatsappOrder: {
    sent: {
      type: Boolean,
      default: false
    },
```

```
    sentAt: Date,
    messageId: String,
    phoneNumber: String
  },
  notes: String,
  couponCode: String,
  couponDiscount: Number,
  ipAddress: String,
  userAgent: String,
  createdAt: {
    type: Date,
    default: Date.now
  },
  updatedAt: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});

// Update stock when order is confirmed
orderSchema.pre('save', async function(next) {
  if (this.isModified('status') && this.status === 'confirmed') {
    const Product = mongoose.model('Product');
    for (const item of this.items) {
      await Product.findByIdAndUpdate(item.productId, {
        $inc: { stock: -item.quantity }
      });
    }
  }
  next();
});

// Indexes
orderSchema.index({ orderId: 1 });
orderSchema.index({ 'customer.phone': 1 });
orderSchema.index({ 'customer.email': 1 });
```

```
orderSchema.index({ status: 1 });
orderSchema.index({ createdAt: -1 });

module.exports = mongoose.model('Order', orderSchema);
```

5. Data Flow

Complete Payment Flow Diagram

```
┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│   User   │ ──────▶ │  Frontend │ ──────▶ │  Backend │ ──────▶ │  MongoDB  │
└──────────────────┘     └──────────────────┘     └──────────────────┘
     │                │                │                │
     │  1. Browse     │                │                │
     │    Products    │                │                │
     │ ◀──────────────────────────────│                │
     │                │                │                │
     │  2. Add to Cart │               │                │
     │ ──────────────────────────────▶│                │
     │                │                │                │
     │  3. Checkout   │                │                │
     │ ──────────────────────────────▶│                │
     │                │  4. Create Order │              │
     │                │ ──────────────────────────────▶│
     │                │                │  5. Save Order │
     │                │                │ ──────────────▶│
     │                │                │ ◀──────────────│
     │                │  6. Order ID   │                │
```

| | | |

| | | |

| 7. Initiate Payment | | |

| 8. Create Razorpay Order | |

| | |

| 9. Razorpay API | | |

| | |

| 10. Order Details | | |

| | |

| 11. Payment UI | | |

| | |

| | |

| 12. Pay with Card/UPI | | |

| | |

| 13. Payment Webhook (Async) | | |

| | |

| | |

| 14. Verify Payment | | |

```
    |──────────────────────────────────────▶| 15. Verify Sig   |
|
    |              |──────────────────────────────────────▶|
|
    |              |                | 16. Update Order |
    |              |                |     Status       |
    |              |
|──────────────────────────────────────▶|
    | 17. Success      |              |              |
    |    Page          |              |              |
    |◀──────────────────────────────────────|              |
|
    |              |              |              |
    | 18. Send WhatsApp (if payment fails)                |

|◀──────────────────────────────────────|──────────────────────────────────────────────────────────────────────────▶|
```

Detailed Flow Steps

Success Path (Razorpay)

1. User adds products to cart
2. User proceeds to checkout and fills details
3. Frontend submits order to backend
4. Backend saves order with `payment.status = 'pending'`
5. Backend creates Razorpay order and returns order_id
6. Frontend loads Razorpay checkout modal
7. User completes payment (card/UPI/netbanking)
8. Razorpay calls frontend success callback
9. Frontend sends verification request to backend
10. Backend verifies signature using Razorpay secret
11. If valid, update order:
    - `payment.status = 'success'`
    - `payment.paidAt = Date.now()`
    - `status = 'confirmed'`
12. Return success to frontend
13. Show order confirmation page with order ID
14. Trigger email confirmation (optional)

15. Update inventory stock levels

Failure Path

1. If payment fails, Razorpay returns error
2. Frontend shows failure message
3. Offer alternative:
   - Retry payment
   - Order via WhatsApp
4. If user chooses WhatsApp:
   - Backend formats order summary
   - Sends message to admin via WhatsApp API
   - Updates order with `whatsappOrder.sent = true`
   - Shows success message with contact instruction

WhatsApp Fallback Message Format

```text
*New Order - AyuraBlend* 🧘

*Order ID:* AYURA-ABC123
*Date:* 15 April 2026

*Customer Details:*
Name: John Doe
Phone: +91 98765 43210
Email: john@example.com

*Shipping Address:*
123, Main Street,
Indiranagar,
Bangalore - 560001
Karnataka

*Order Items:*
━━━━━━━━━━━━━━━━━━━━

1. Lavender Essential Oil
   Quantity: 2 × ₹499 = ₹998

2. Wellness Kit
   Quantity: 1 × ₹2499 = ₹2499
━━━━━━━━━━━━━━━━━━━━

*Subtotal:*    ₹3497
*Shipping:*    ₹0
```

```
*Tax (GST):*   ₹629
*Total:*       ₹4126

*Payment Method:* To be confirmed via WhatsApp

Please contact customer to complete the order.

🔗 View Order: https://ayurablend.com/orders/ABC123
```

6. React Component Structure

Component Hierarchy

```text
App (Root)
│
├── Layout
│   ├── Navbar
│   │   ├── Logo
│   │   ├── NavigationLinks
│   │   │   ├── HomeLink
│   │   │   ├── ProductsLink
│   │   │   ├── CategoriesDropdown
│   │   │   └── ContactLink
│   │   ├── SearchBar
│   │   └── CartIcon
│   │       └── CartBadge (count)
│   │
│   ├── MainContent
│   │   ├── Routes
│   │   │   ├── HomePage
│   │   │   │   ├── HeroSection
│   │   │   │   │   ├── Carousel
│   │   │   │   │   └── CTAButton
│   │   │   │   ├── FeaturedProducts
│   │   │   │   │   └── ProductCarousel
│   │   │   │   ├── CategoryGrid
│   │   │   │   │   └── CategoryCard (x4)
│   │   │   │   ├── BenefitsSection
│   │   │   │   └── NewsletterSignup
│   │   │   │
│   │   │   ├── ProductsPage
│   │   │   │   ├── FiltersSidebar
│   │   │   │   │   ├── CategoryFilter
│   │   │   │   │   ├── PriceRangeFilter
│   │   │   │   │   ├── RatingFilter
```

```
│   │   │   │   │   │   │   └── AvailabilityFilter
│   │   │   │   │   │   ├── ProductGrid
│   │   │   │   │   │   │   └── ProductCard (multiple)
│   │   │   │   │   │   │       ├── ProductImage
│   │   │   │   │   │   │       ├── ProductBadge (sale/new)
│   │   │   │   │   │   │       ├── ProductTitle
│   │   │   │   │   │   │       ├── ProductPrice
│   │   │   │   │   │   │       ├── RatingStars
│   │   │   │   │   │   │       └── QuickAddButton
│   │   │   │   │   │   └── Pagination
│   │   │   │   │   │
│   │   │   │   │   ├── ProductDetailPage
│   │   │   │   │   │   ├── ProductGallery
│   │   │   │   │   │   │   ├── MainImage
│   │   │   │   │   │   │   └── ThumbnailList
│   │   │   │   │   │   ├── ProductInfo
│   │   │   │   │   │   │   ├── ProductTitle
│   │   │   │   │   │   │   ├── ProductRating
│   │   │   │   │   │   │   ├── ProductPrice
│   │   │   │   │   │   │   ├── StockStatus
│   │   │   │   │   │   │   ├── QuantitySelector
│   │   │   │   │   │   │   ├── AddToCartButton
│   │   │   │   │   │   │   └── BuyNowButton
│   │   │   │   │   │   └── ProductTabs
│   │   │   │   │   │       ├── DescriptionTab
│   │   │   │   │   │       ├── BenefitsTab
│   │   │   │   │   │       ├── IngredientsTab
│   │   │   │   │   │       ├── HowToUseTab
│   │   │   │   │   │       └── ReviewsTab
│   │   │   │   │   │
│   │   │   │   │   ├── CartPage
│   │   │   │   │   │   ├── CartItemsList
│   │   │   │   │   │   │   └── CartItem (multiple)
│   │   │   │   │   │   │       ├── ItemImage
│   │   │   │   │   │   │       ├── ItemDetails
│   │   │   │   │   │   │       ├── QuantityControls
│   │   │   │   │   │   │       ├── ItemPrice
│   │   │   │   │   │   │       └── RemoveButton
│   │   │   │   │   │   ├── CartSummary
│   │   │   │   │   │   │   ├── Subtotal
│   │   │   │   │   │   │   ├── Shipping (calculated)
│   │   │   │   │   │   │   ├── Tax (calculated)
│   │   │   │   │   │   │   ├── Total
│   │   │   │   │   │   │   └── PromoCodeInput
│   │   │   │   │   │   └── CheckoutButton
│   │   │   │   │   │
│   │   │   │   │   ├── CheckoutPage
│   │   │   │   │   │   ├── MultiStepForm
```

```
│   │   │   │   │   │   ├── Step1_CustomerInfo
│   │   │   │   │   │   │   ├── NameInput
│   │   │   │   │   │   │   ├── EmailInput
│   │   │   │   │   │   │   └── PhoneInput
│   │   │   │   │   │   ├── Step2_AddressForm
│   │   │   │   │   │   │   ├── AddressLine1
│   │   │   │   │   │   │   ├── AddressLine2
│   │   │   │   │   │   │   ├── CitySelect
│   │   │   │   │   │   │   ├── StateSelect
│   │   │   │   │   │   │   └── PincodeInput
│   │   │   │   │   │   └── Step3_PaymentMethod
│   │   │   │   │   │       ├── RazorpayOption
│   │   │   │   │   │       ├── PayPalOption
│   │   │   │   │   │       └── WhatsAppOption
│   │   │   │   │   ├── OrderSummarySidebar
│   │   │   │   │   └── PaymentButton
│   │   │   │   │
│   │   │   │   ├── OrderSuccessPage
│   │   │   │   │   ├── SuccessIcon
│   │   │   │   │   ├── OrderDetails
│   │   │   │   │   ├── OrderItemsList
│   │   │   │   │   ├── ShareButtons
│   │   │   │   │   │   ├── WhatsAppShare
│   │   │   │   │   │   └── EmailShare
│   │   │   │   │   └── ContinueShoppingButton
│   │   │   │   │
│   │   │   │   └── OrderFailurePage
│   │   │   │       ├── ErrorIcon
│   │   │   │       ├── ErrorMessage
│   │   │   │       ├── RetryPaymentButton
│   │   │   │       ├── WhatsAppOrderButton
│   │   │   │       └── ContactSupport
│   │   │   │
│   │   │   └── CartDrawer (slide-out panel)
│   │   │       ├── CartItems (mini)
│   │   │       ├── Subtotal
│   │   │       ├── ViewCartButton
│   │   │       └── CheckoutButton
│   │   │
│   │   └── Footer
│   │       ├── CompanyInfo
│   │       ├── QuickLinks
│   │       ├── ContactInfo
│   │       ├── SocialLinks
│   │       └── NewsletterSignup
```

Component State Management

CartContext Structure:

```javascript
{
  cart: [],
  addToCart: (product, quantity) => void,
  removeFromCart: (productId) => void,
  updateQuantity: (productId, quantity) => void,
  clearCart: () => void,
  getCartTotal: () => number,
  getCartCount: () => number,
  isInCart: (productId) => boolean
}
```

OrderContext Structure:

```javascript
{
  currentOrder: null,
  createOrder: (orderData) => Promise,
  updateOrderStatus: (orderId, status) => Promise,
  sendWhatsAppOrder: (orderId) => Promise,
  clearCurrentOrder: () => void
}
```

7. Step-by-Step Build Plan

Phase 1: Setup & Infrastructure (Week 1 - 2 days)

Module 1: Project Initialization

Duration: 1 day

Tasks:

- Create frontend project: `npm create vite@latest ayurablend-frontend -- --template react`
- Install Tailwind CSS and configure:
  ```bash
  npm install -D tailwindcss postcss autoprefixer
  npx tailwindcss init -p
  ```

- Install additional frontend dependencies:
```bash
npm install react-router-dom axios react-hook-form
npm install @headlessui/react heroicons react-hot-toast
```
- Create backend project structure:
```bash
mkdir ayurablend-backend && cd ayurablend-backend
npm init -y
```
- Install backend dependencies:
```bash
npm install express mongoose dotenv cors bcryptjs jsonwebtoken
npm install express-validator helmet morgan compression
npm install cloudinary multer razorpay
npm install axios nodemailer
npm install -D nodemon
```
- Setup folder structure as per Section 2
- Configure environment variables (.env files)
- Setup Git repository with .gitignore

Module 2: Database Setup & Models

Duration: 1 day

Tasks:

- Create MongoDB Atlas cluster (free tier)
- Configure network access (allow all IPs temporarily)
- Create database user with read/write permissions
- Implement `db.js` connection utility with retry logic
- Create Product model with indexes (as per Section 4)
- Create Order model with pre-save hooks (as per Section 4)
- Create seed script to insert sample products (5-10 products)
- Test database connection and CRUD operations
- Setup MongoDB Compass for visual management

Sample Seed Data:

```javascript
// seed.js
const products = [
  {
    name: "Lavender Essential Oil",
    slug: "lavender-essential-oil",
    description: "100% pure lavender essential oil for relaxation...",
    price: 499,
    category: "essential-oils",
    stock: 50,
    images: [{ url: "https://res.cloudinary.com/..." }]
  },
  // Add 9 more products
];
```

Phase 2: Core Backend APIs (Week 1 - 2 days)

Module 3: Products API

Duration: 1 day

Tasks:

- Create `productController.js` with CRUD functions
- Implement GET `/api/products` with:
  - Pagination (limit/offset)
  - Category filtering
  - Price range filtering
  - Search functionality
  - Sorting options
- Implement GET `/api/products/:id`
- Implement GET `/api/products/category/:category`
- Add response caching for product list
- Create validation middleware for product data
- Test all endpoints with Postman/Insomnia

Module 4: Orders API

Duration: 1 day

Tasks:

- Create `orderController.js`
- Implement POST `/api/orders` with:
  - Request validation
  - Stock checking before order creation
  - Automatic orderId generation
  - Save to database
- Implement GET `/api/orders/:id`
- Implement GET `/api/orders/phone/:phone` (for order lookup)
- Add rate limiting to prevent abuse
- Create order status update endpoint
- Add order history tracking

Phase 3: Frontend Foundation (Week 2 - 3 days)

Module 5: Frontend Setup & Routing

Duration: 1 day

Tasks:

- Setup React Router with routes:
  javascript

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/products" element={<ProductsPage />} />
  <Route path="/product/:slug"
element={<ProductDetailPage />} />
  <Route path="/cart" element={<CartPage />} />
  <Route path="/checkout" element={<CheckoutPage />} />
  <Route path="/order-success" element={<OrderSuccessPage
/>} />
  <Route path="/order-failure" element={<OrderFailurePage
/>} />
</Routes>
```

- Create Layout component with Navbar and Footer
- Configure Axios instance:
  javascript

```
const api = axios.create({
```

```
  baseURL: import.meta
```